

9/7일 과제 3 C++ 분석 보고서

namespace : 이름 공간 생성 및 사용

namespace <사용자 이름> {hpp에 선언하는 내용}

여러 사람들과 프로그램을 개발할 때 사용하는 함수나 변수의 이름이 충돌하는 경우가 있다. 이를 해결하기 위한 것이 **namespace**이다. **namespace** 사용법은 hpp에서 클래스/구조체 선언 전체를 **namespace** <namespace 이름> {...}로 감싸고 cpp에서 사용할 때는 <namespace 이름> :: <class 이름> :: 함수이름, 형태로 쓰거나 cpp 전체를 **namespace** <namespace 이름> {}로 감싸면 된다. 지금은 과제를 위해서 프로그램을 작성하지만 추후에 코드를 작성할 때 다른 개발자와 공유할 것을 생각하면 **namespace**를 사용하는 버릇을 들여놓는 것이 좋을 것 같다고 생각했다.

basic-reference : 참조자

`int num1 = 10; int& num2 = num1;`을 하게 되면 `num2`가 `num1`을 참조하는 상태가 됨. 참조를 하게 되면 `num1 == num2`, `&num1 == &num2`인 상태가 됨. 즉 `num2`가 `num1`을 치환한다고 생각하면 이해하기 쉽고 그렇기 때문에 포인터처럼 따로 변수를 생성하는 것을 아니라 새로운 메모리 공간이 할당되지는 않음. C나 C++에서 함수의 반환값은 1개이기 때문에 여러개의 반환값을 내고 싶을 때에는 포인터를 사용하였다. 그런데 참조자를 이용하면 굳이 포인터를 사용하지 않고도 원본값을 수정할 수 있다는 것을 알았다. 다음부터는 참조자를 이용해서 더 편리하게 코드를 작성할 것이다.

new, delete : 동적할당

C++에서는 C의 **malloc**과 다르게 동적할당을 한다. C++에서는 **new**로 동적할당 메모리를 만들고, **delete**로 동적할당 메모리를 지운다. 사용하는 방법은 **new** <만들자료형>; 예를 들어 `int count = 10; int* arr = new int[count];`로 `arr`를 동적할당할 수 있다. 그리고 **delete** <없앨 메모리 포인터>; 예를 들어 `delete[] arr;`을 하면 `arr` 메모리를 해제할 수 있다. 사용해보니 **new**가 **malloc**보다 사용하기 편해서 앞으로 **new**를 계속 사용할 것 같다.

struct 구조체

과제 2를 하면서 `_Point` 구조체를 만들었는데 이 구조체는 hpp에서만 선언해놓았다. 그래서 cpp에서는 구조체를 사용할 수 없을 것 같아서 cpp에도 `_Point` 구조체를 선언하였다. 그런데 에러가 발생해서 보니 hpp에 **struct**가 있을 때 **include**하면 그 구조체를 그대로 사용할 수 있다는 것을 알게 되었다.

cout 과 **endl**

예제를 공부하면서 **endl**이 줄바꿈을 해준다는 것을 알게 되었다. 그리고 과제 1의 평균을 출력했을 때 소숫점 아래 .00000이 출력되지 않아 자료형을 잘못 설정했나 살펴보니 **cout**이 자동으로 소수점 아래가 0000 이면 버려준다는 것을 알게 되었다.

random : 난수 생성

과제 2를 풀면서 c++에 여러 난수 생성 방법이 있다는 것을 알게 되었다. c++에서는 c에서처럼 **rand()** 와 **srand()**로 무작위한 난수를 생성할 수 있다. 문법은 살짝 다르다. **srand**로 시드값을 변경하려면 **srand(static_cast<unsigned int>(time(NULL)))**;을 사용하면 된다. 그런데 이렇게 단순히 시드값을 초기화해주는 것은 난수의 품질이 좋지 않을 수 있다고 한다. 그래서 c++에서는 **random** 라이브러리에서 다양한 난수 생성 알고리즘을 제공한다. 그중 **Mersenne Twister** 알고리즘 사용방법은

`std::default_random_engine_generator; std::uniform_int_distribution<자료형> distribution(<min>, <max>)`이다. 하지만 이러면 계속해서 같은 난수가 나온다. 그래서 과제2에서 사용한 난수 클래스는 `random_device`이다. 사용법은 `std::random_device rd; std::uniform_int_distribution<자료형> distribution(min, max);`이다. 이렇게하면 실행시킬 때마다 다른 값이 나오고 품질이 좋은 난수를 생성할 수 있다. 그래서 추후에 난수를 생성할 일이 있으면 이 방식을 사용하려고 한다.

class

클래스는 구조체에서 확장되었다고 생각했을 때 이해하기 편했다. 구조체와 차이로는 메모로 함수를 포함 할 수 있다는 것과 데이터 접근에 제한을 둘 수 있다는 것이다. 클래스를 사용할 때는 크게 선언부와 구현부로 나눌 수 있다. 보통 선언부는 `hpp`에 저장되며 주로 선언만 한다. 이때 `private:`과 `public:`으로 나누어 변수와 함수의 접근성을 정할 수 있다. `private`은 클래스 내부에서만 수정할 수 있고, `public`은 `class` 외부에서도 수정이 가능하다. `class` 구현부는 선언한 클래스 내의 함수의 내용을 구현한다. `c`에서 함수를 만들 때 위에 `void func();`으로 선언만 하고 이후에 구현하는 것을 생각하면 이해하기 쉬웠다. 클래스 내부의 함수를 구현 할 때에는 `<자료형> <클래스 이름>::<함수 이름>() {...}` 이런식으로 구현하면 된다.

class Overloading

오버로딩은 같은 이름의 함수에 입력받는 매개변수를 달리하여 함수 호출을 잘못하는 실수를 줄일 수 있는 문법이다. 즉 함수를 중복 시키는 건데 이때 조건은 중복된 함수들의 이름이 같아야하고 함수들의 매개 변수 타입이 다르거나 개수가 달라야한다. 그리고 이때 리턴 타입은 함수 중복과 무관하기 때문에 이름이 같으면 된다고 보면 된다.

class 생성자, 소멸자

생성자는 객체(`class`)가 생성되는 시점에 자동으로 호출되는 멤버 함수이다. 생성자의 목적은 객체가 생성될 때 객체가 필요한 초기화를 위해 생성된다. 생성자를 구현 할 때에는 `<클래스 이름>::<클래스 이름> {...}` 이렇게 하면된다. 그리고 당연히 오버로딩이 가능하다. 소멸자는 객체가 소멸되는 시점에 자동으로 호출되는 함수이다. 소멸자의 목적은 객체가 사라질 때 마무리 작업을 하기 위함이다. 예를 들어 객체가 생성된 후, 소멸 되기 전에 동적 메모리를 할당해 놓았다면 소멸 단계에서 메모리를 해제하기 위함이다.

추가 과제

과제 3을 작성하면서 `namespace` 작성, 참조자 에 대해 정리하고 나서 다시 과제2를 봤을 때 수정해야될 부분이 있다는 생각이 들었다. 먼저 과제2에서는 `namespace`를 사용하지 않았다. 과제 3을 하면서 `namespace`가 있는게 좋다고 생각했고 습관화하기로 했기에 수정하고자 하였다. 그리고 과제2를 풀 당시에는 참조자가 익숙하지 않아서 그냥 포인터로 저장한 값을 저장했는데 참조자를 사용하면 굳이 그럴필요도 없고 코드가 더 깔끔해질 수 있다는 것을 깨달아 코드를 수정하고자 하였다. 그리고 과제 2 코드를 다시 보니 값 입력받고 출력받는 것도 클래스의 멤버 함수로 역할을 분리할 수 있는데 습관적으로 `main`에 욱여넣은 것을 발견했고 이 부분도 함수화하는 것이 좋을 것 같고 함수화를 습관화해서 `main`을 최대한 깔끔하게 만드는것이 좋을 것 같아 코드를 수정하고자 하였다.

```
void Point::calcPointDist(int size, _Point* arr, double* MinDist, double* MaxDist, _Point* minPoint1, _Point* minPoint2, _Point* maxPoint1, _Point* maxPoint2)
```

이 부분을

```
void Point::calcPointDist(int size, _Point* arr, double& MinDist, double& MaxDist, _Point& minPoint1, _Point& minPoint2, _Point& maxPoint1, _Point& maxPoint2)
```

이렇게 바꿔서 참조로 바꾸고

```
p.calcPointDist(num, p.arr, Mindist, Maxdist, minPoint1, minPoint2, maxPoint1, maxPoint2);
```

호출부에서 값을 넣어주어 참조하도록 하였다.

그리고 `getInput()` 함수로 입력받을 정보를 출력하고 사용자로 부터 그 정보를 입력받는다.

그리고 `printResult()`를 이용해서 계산한 결과값을 출력하는 함수를 따로 만들었다. 이를

통해 `main`문을 간소화 시켰다.

`hpp`에서 내가 작성한 `class`를 `namespace`로 묶었고 `cpp`에서 내가 만든 클래스 앞에 `yep::`을 붙여서 `namespace`를 활용해 보았다. 코드를 바꿔보면서 `hpp`의 구조체에 `namespace`를 넣었을 때도 `cpp`에서 사용할 수 없다는 것을 알았다. 그런데 `_Point` 앞에 `yep::`을 붙이면 이 문제를 해결할 수 있다는 것도 처음 알게 되었다.

```
***** HW 3 Point Distance Computation *****

Please define the number of points(number > 1) : 8
Please define minimum of coord. value : 9
Please define maximum of coord. value : 25
Generate Random points

Point 0. nX=23 , nY=12
Point 1. nX=16 , nY=10
Point 2. nX=19 , nY=14
Point 3. nX=23 , nY=23
Point 4. nX=12 , nY=9
Point 5. nX=12 , nY=18
Point 6. nX=10 , nY=14
Point 7. nX=21 , nY=15

----- Result -----
MinDist=2.23607
Pair of Min Coord.<x, y>: P1<19,14> & P2<21,15>

MaxDist=17.8045
Pair of Max Coord.<x, y>: P1<23,23> & P2<12,9>

***** Completed *****
```

수정한 코드에서도 정상적으로 작동하는 것을 확인하였다.